# EduGenius AI

## RAG-Powered Presentation Generator

Complete Technical Documentation

File-by-File README with Detailed Logic

24 Files Documented

*Projet de Fin d'Etudes (PFE)*

# Table of Contents

1. Global System Overview
2. Project Structure (Files to Keep vs Remove)
3. Root-Level Files Documentation
4. Modules Documentation (21 files)
5. Complete End-to-End Flow
6. Key Technical Concepts
7. API Endpoints Reference

# 1. Global System Overview

EduGenius AI is a RAG (Retrieval-Augmented Generation) system that transforms PDF documents into professional educational presentations. The system combines document processing, semantic search, and LLM generation to create slide decks automatically.

## 1.1 What Problem Does It Solve?

Teachers and educators spend hours manually creating presentations from textbooks and documents. This system automates that process: upload a PDF, type a topic, and get a complete presentation in seconds.

## 1.2 High-Level Architecture

The system follows a 10-step pipeline:

1. PDF Ingestion - Extract text and images from uploaded documents
2. OCR Processing - Handle scanned pages with EasyOCR/Tesseract
3. Semantic Chunking - Split text into meaningful pieces (512 chars)
4. Embedding - Convert chunks to 768-dim vectors (BAAI/bge-base-en-v1.5)
5. Indexing - Store in FAISS vector database + BM25 keyword index
6. Hybrid Retrieval - FAISS + BM25 + RRF fusion + Cross-encoder reranking
7. LLM Generation - Groq API (primary) or Ollama (fallback) creates slides
8. Quality Control - Self-scoring + optional LLM-as-judge
9. Diagram Generation - Mermaid diagrams based on visual hints
10. HTML Rendering - Professional themed output with animations

## 1.3 Key Technologies

| Component | Technology |
|---|---|
| Backend | FastAPI (Python async) |
| Embeddings | SentenceTransformers (BAAI/bge-base-en-v1.5) |
| Vector DB | FAISS (IndexFlatIP) |
| Keyword Search | BM25Okapi (rank_bm25) |
| Reranker | CrossEncoder (BAAI/bge-reranker-base) |
| LLM | Groq API (llama-3.3-70b) / Ollama (local fallback) |
| OCR | EasyOCR / Tesseract |
| PDF Processing | PyMuPDF (fitz) |

# 2. Project Structure

Here is the recommended clean project structure after removing unnecessary files:

## 2.1 Files to KEEP (24 total)

### Root Level (3 files)

- api.py - Main FastAPI application (orchestrates everything)
- config.yaml - Central configuration file
- rebuild_index.py - CLI tool to rebuild FAISS index

### modules/ Folder (21 files)

- __init__.py - Package marker
- config_loader.py - Configuration manager
- ingestion.py - PDF/image loading
- ocr.py - OCR processing
- text_processing.py - Semantic chunking
- embeddings.py - Vector creation + FAISS
- retrieval.py - Hybrid search
- query_expansion.py - Multi-query retrieval
- llm.py - LLM engine (Groq/Ollama)
- llm_cache.py - Response caching
- retry_utils.py - Retry logic
- context_manager.py - Context preparation
- pedagogical_engine.py - Slide generation
- schemas.py - Pydantic validation
- slide_generator.py - SlideData conversion
- diagram_generator.py - Mermaid diagrams
- visual_strategy.py - Visual assignment
- html_renderer.py - HTML output
- evaluation.py - RAG metrics
- health.py - System monitoring
- history_store.py - Presentation history

## 2.2 Files to REMOVE

| File | Reason for Removal |
|---|---|
| test_llm_cache.py | Development test file - not needed for production |
| test_retry.py | Development test file - not needed for production |

# 3. Root-Level Files Documentation

## api.py

| | |
|---|---|
| **Role** | Main FastAPI application. The entry point that orchestrates the entire RAG pipeline. Handles HTTP requests, file uploads, and SSE streaming for real-time progress. |
| **Before** | HTTP requests from React frontend: file uploads via multipart/form-data, generation requests with prompt/theme/num_slides parameters, history and health queries. |
| **Inside** | • FastAPI app initialization with CORS middleware for frontend communication<br>• CORS allows origins: localhost:5173, 8080-8090 for development<br>• @app.on_event('startup'): Pre-warms OCR engine, embedding model, and reranker at server start<br>• _files_hash(): Creates SHA-1 hash of uploaded files for caching<br>• _build_image_registry(): Extracts images from PDFs, assigns simple IDs (IMG_001, IMG_002)<br>• _assign_fallback_images(): Auto-matches images to slides using word overlap scoring<br>• POST /generate-stream: Main endpoint with SSE streaming:<br>• - Creates temp directory and session UUID<br>• - Saves uploaded files to raw/ subdirectory<br>• - Calls ingest_directory() to extract text/images<br>• - Runs OCR on scanned pages<br>• - Chunks text with process_pages()<br>• - Builds FAISS index with build_vector_db()<br>• - Creates Retriever and searches for relevant chunks<br>• - BATCH MODE: Generates ALL slides in one LLM call (5x faster)<br>• - SEQUENTIAL MODE: One LLM call per slide with quality gate<br>• - Generates Mermaid diagrams based on visual_hint<br>• - Renders final HTML with theme and animations<br>• - Records presentation to history store<br>• - Emits SSE events: status, slide, done, error<br>• GET /view/{session_id}: Serves generated HTML presentations<br>• GET /history: Returns all past presentations as JSON<br>• DELETE /history: Clears presentation history<br>• GET /health: Full system health check (all components)<br>• GET /health/quick: Lightweight status for polling<br>• GET /cache/stats: LLM cache statistics<br>• DELETE /cache: Clears LLM response cache<br>• POST /evaluate: Evaluates RAG quality metrics for a query |
| **After** | StreamingResponse with SSE events for real-time progress. Final HTML saved to outputs/ directory and served via /view/{session_id} endpoint. |

## config.yaml

| | |
|---|---|
| **Role** | Central configuration file. Single source of truth for all system parameters. Loaded once at startup and cached as singleton. |
| **Before** | Static YAML file in project root. Edited manually or via deployment scripts. |
| **Inside** | • project: name, version, description metadata<br>• paths section:<br>• - data_raw: 'data/raw' - where PDFs are uploaded |

| | |
|---|---|
| | • - data_processed: 'data/processed' - FAISS index location<br>• - outputs: 'outputs' - generated presentations<br>• ocr section:<br>• - engine: 'easyocr' (or 'tesseract')<br>• - languages: ['fr', 'en'] - supported languages<br>• - device: 'cuda' or 'cpu'<br>• text_processing section:<br>• - chunk_size: 512 characters per chunk<br>• - chunk_overlap: 64 characters overlap between chunks<br>• embeddings section:<br>• - model: 'BAAI/bge-base-en-v1.5' - embedding model name<br>• - device: 'cuda' or 'cpu'<br>• vector_db section:<br>• - backend: 'faiss'<br>• - index_path: 'data/processed/faiss_index'<br>• retrieval section:<br>• - use_bm25: true - enable hybrid search<br>• - use_reranker: true - enable cross-encoder<br>• - reranker_model: 'BAAI/bge-reranker-base'<br>• - reranker_top_n: 5 - candidates to rerank<br>• llm section:<br>• - api_url: 'http://127.0.0.1:11434/api/generate' (Ollama)<br>• - model: 'llama3.2'<br>• - temperature: 0.7<br>• - context_length: 2048<br>• - judge_model: 'mistral' - for quality scoring<br>• - judge_enabled: false - disable to save time<br>• generation section:<br>• - quality_threshold: 5 - minimum score (1-10)<br>• - parallel: false - sequential to save memory<br>• pdf_images section:<br>• - extract: true - enable image extraction<br>• - max_per_doc: 20 - limit per PDF |
| **After** | Loaded by config_loader.py as singleton CONFIG dict. Imported by all modules via: from modules.config_loader import CONFIG |

## rebuild_index.py

| | |
|---|---|
| **Role** | CLI utility script to rebuild the FAISS vector index. Run this after adding new PDFs to data/raw/ directory. |
| **Before** | PDF files placed in data/raw/ directory. User runs: python rebuild_index.py |
| **Inside** | • Step 1 - Ingestion:<br>• - Reads CONFIG['paths']['data_raw'] path<br>• - Calls ingest_directory() from ingestion.py<br>• - Extracts text and images from all PDFs<br>• - Prints count of pages loaded<br>• Step 2 - OCR:<br>• - Calls run_ocr() from ocr.py |

| | |
|---|---|
| | • - Processes image pages through EasyOCR<br>• - Populates text field for scanned pages<br>• Step 3 - Chunking:<br>• - Calls process_pages() from text_processing.py<br>• - Splits text into semantic chunks<br>• - Respects paragraph/sentence boundaries<br>• - Prints count of chunks created<br>• Step 4 - Embedding & Indexing:<br>• - Calls build_vector_db() from embeddings.py<br>• - Loads SentenceTransformer model<br>• - Encodes all chunks to 768-dim vectors<br>• - Creates FAISS IndexFlatIP<br>• - Saves .index, .json, .bm25.json files<br>• Prints success message with instructions |
| **After** | Updated index files in data/processed/: faiss_index.index (FAISS binary), faiss_index.json (chunk metadata), faiss_index.bm25.json (tokenized corpus for BM25). Index ready for retrieval. |

# 4. Modules Documentation

Each module in the modules/ folder is documented with detailed bullet points explaining the internal logic.

## config_loader.py

| | |
|---|---|
| **Role** | Configuration manager module. Loads config.yaml once and provides a singleton CONFIG dict to all other modules. |
| **Before** | config.yaml file in project root containing all system settings. |
| **Inside** | • Imports yaml library for YAML parsing<br>• Resolves project root path using Path(__file__).resolve().parent<br>• Handles both running from modules/ and from root directory<br>• load_config() function:<br>• - Opens config.yaml with UTF-8 encoding<br>• - Uses yaml.safe_load() for secure parsing<br>• - Returns configuration as Python dict<br>• CONFIG singleton:<br>• - Loaded once at module import time<br>• - Reused across all imports (no re-reading)<br>• Environment variable overrides:<br>• - OLLAMA_URL overrides llm.api_url<br>• - OLLAMA_MODEL overrides llm.model<br>• CUDA fallback logic:<br>• - If device='cuda' but torch.cuda.is_available() is False<br>• - Automatically switches to device='cpu'<br>• - Prevents runtime errors on non-GPU machines<br>• CLI test mode: prints JSON config when run directly |
| **After** | CONFIG dict exported as module-level variable. All modules import via: from modules.config_loader import CONFIG |

## ingestion.py

| | |
|---|---|
| **Role** | Document ingestion module. Loads PDFs, images, and text files from data/raw/ directory. Extracts text and embedded images. |
| **Before** | Raw document files in data/raw/: PDFs (.pdf), images (.png, .jpg, .jpeg, .bmp, .tiff, .webp), text files (.txt) |
| **Inside** | • DocumentPage dataclass:<br>• - source: filename string<br>• - page: page number (1-based for PDFs, 0 for images)<br>• - type: 'pdf' \| 'image' \| 'txt' \| 'pdf_image'<br>• - text: extracted text content<br>• - image: base64-encoded image data<br>• load_pdf() function:<br>• - Opens PDF with PyMuPDF (fitz.open)<br>• - Iterates through all pages<br>• - Extracts text using page.get_text('text')<br>• - Creates DocumentPage for each text page<br>• - Diagram detection: if text < 500 chars AND drawings > 2 |

| | |
|---|---|
| | • - Rasterizes full page at 120 DPI<br>• - Converts to base64 JPEG<br>• - Creates 'pdf_image' type DocumentPage<br>• - Embedded image extraction:<br>• - Gets image list with page.get_images()<br>• - Extracts first image per page<br>• - Skips tiny images (< 100x100 pixels)<br>• - Converts to base64 JPEG<br>• - Closes document after processing all pages<br>• load_image() function:<br>• - Opens image with PIL<br>• - Converts to RGB mode<br>• - Encodes to base64 JPEG<br>• load_txt() function:<br>• - Reads file with UTF-8 encoding<br>• - Handles encoding errors gracefully<br>• ingest_directory() function:<br>• - Lists all files in raw_dir<br>• - Skips .gitkeep files<br>• - Routes to appropriate loader by extension<br>• - Returns combined list of DocumentPage objects |
| **After** | List of DocumentPage objects containing: source filename, page number, type, text content, and optional base64 image. Passed to ocr.py for scanned pages or text_processing.py for chunking. |

## ocr.py

| | |
|---|---|
| **Role** | OCR (Optical Character Recognition) module. Extracts text from scanned documents and images using neural networks. |
| **Before** | DocumentPage objects with type='image' and empty text field. These are standalone images or scanned PDF pages. |
| **Inside** | • Global cache: _EASYOCR_CACHE stores loaded model<br>• warm_ocr_engine() function:<br>• - Called at server startup (@app.on_event)<br>• - Pre-loads EasyOCR model into memory<br>• - Saves 3-5 seconds on first request<br>• - Respects device config (cuda/cpu)<br>• OCREngine class:<br>• - __init__: reads engine type from config<br>• - Supports 'easyocr' and 'tesseract' backends<br>• - Languages from config: ['fr', 'en']<br>• _initialize() method:<br>• - For EasyOCR:<br>• - Checks global cache first<br>• - Creates easyocr.Reader with language list<br>• - GPU enabled if device='cuda'<br>• - For Tesseract:<br>• - Imports pytesseract library<br>• - Maps language codes (fr->fra, en->eng) |

| | |
|---|---|
| | • process_image() method:<br>• - Accepts PIL Image or base64 string<br>• - If base64: decodes and opens as PIL Image<br>• - For EasyOCR:<br>• - Converts to numpy array<br>• - Calls reader.readtext(detail=0)<br>• - Joins results with newlines<br>• - For Tesseract:<br>• - Calls image_to_string() with language<br>• run_ocr() function:<br>• - Filters pages needing OCR (type='image', has image)<br>• - Creates OCREngine instance<br>• - Processes each image page<br>• - Updates page.text with OCR result |
| **After** | Same DocumentPage objects but with text field populated from OCR. Ready for text_processing.py to chunk into smaller pieces. |

## text_processing.py

| | |
|---|---|
| **Role** | Semantic-aware text chunking module. Splits documents into meaningful pieces that preserve context for embedding and retrieval. |
| **Before** | List of DocumentPage objects with text content from ingestion or OCR. |
| **Inside** | • TextChunk dataclass:<br>• - text: chunk content string<br>• - source: original filename<br>• - page: page number<br>• - chunk_id: unique sequential ID<br>• clean_text() function:<br>• - Fixes hyphenation at line breaks (re-joins split words)<br>• - Normalizes whitespace (max 2 newlines)<br>• - Collapses multiple spaces to single space<br>• _split_into_paragraphs() function:<br>• - Splits on double newlines (paragraph breaks)<br>• - Strips whitespace from each paragraph<br>• _split_into_sentences() function:<br>• - Preserves decimals (1.5 -> 1<DOT>5)<br>• - Handles abbreviations (Mr., Dr., etc.)<br>• - Splits on sentence endings (.!?)<br>• - Restores dots after splitting<br>• _is_list_item() function:<br>• - Detects bullet points (-, *, •)<br>• - Detects numbered lists (1., 2))<br>• - Detects lettered lists (a., b))<br>• _find_list_block() function:<br>• - Finds consecutive list items<br>• - Returns end index of list block<br>• semantic_chunk() function:<br>• - Phase 1: Groups paragraphs semantically<br>• - Keeps list items together as blocks<br>• - Phase 2: Splits large blocks, merges small ones<br>• - If block fits in current chunk: append<br>• - If block too large: split at sentence boundaries<br>• - If block doesn't fit: start new chunk<br>• - Minimum chunk size: 100 chars<br>• - Phase 3: Adds overlap for context<br>• - Takes last N chars from previous chunk<br>• - Finds clean word boundary<br>• - Prepends with '...' separator<br>• process_pages() function:<br>• - Reads chunk_size and chunk_overlap from config<br>• - Iterates through all pages with text<br>• - Calls semantic_chunk() for each page<br>• - Assigns sequential chunk_id to each TextChunk |
| **After** | List of TextChunk objects with: text (chunk content), source (filename), page (number), chunk_id (sequential). Passed to embeddings.py for vectorization. |

## embeddings.py

| | |
|---|---|
| **Role** | Vector embedding and FAISS index builder. Converts text chunks into numerical vectors and stores them for similarity search. |
| **Before** | List of TextChunk objects from text_processing.py. |
| **Inside** | • Global cache: _GLOBAL_EMBED_MODEL for model singleton<br>• Global cache: _INDEX_CACHE for index caching by content hash<br>• _chunks_hash() function:<br>• - Creates SHA-256 hash of all chunk texts<br>• - Used as cache key for index<br>• VectorDB class:<br>• - __init__: reads model name, device, index_path from config<br>• _load_model() method:<br>• - Checks global cache first<br>• - Loads SentenceTransformer with model name<br>• - Stores in global for reuse<br>• embed_and_store() method:<br>• - Cache check: if hash matches, return cached index<br>• - Loads embedding model<br>• - Extracts text list from chunks<br>• - Encodes all texts: model.encode()<br>• - Converts to float32 numpy array<br>• - L2 normalization for cosine similarity:<br>• - faiss.normalize_L2(embeddings)<br>• - Semantic deduplication (if <= 500 chunks):<br>• - Computes similarity matrix<br>• - Drops chunks with cosine > 0.95<br>• - Logs number of duplicates removed<br>• - Creates FAISS index:<br>• - IndexFlatIP for inner product (cosine)<br>• - Adds all normalized vectors<br>• - Updates global cache<br>• _save_to_disk() method:<br>• - Creates output directory if needed<br>• - Saves FAISS index: faiss.write_index()<br>• - Saves chunk metadata as JSON<br>• - Saves BM25 corpus: tokenized lowercase words<br>• build_vector_db() function:<br>• - Creates VectorDB instance<br>• - Calls embed_and_store() |
| **After** | Three files saved to data/processed/: faiss_index.index (FAISS binary), faiss_index.json (chunk metadata with text/source/page), faiss_index.bm25.json (tokenized corpus for BM25). Ready for retrieval.py to load and search. |

## retrieval.py

| | |
|---|---|
| **Role** | Hybrid retrieval engine. Combines semantic search (FAISS) with keyword search (BM25) and cross-encoder reranking for accurate results. |
| **Before** | User query string. Pre-built FAISS index and BM25 corpus loaded from disk. |
| **Inside** | • Global cache: _GLOBAL_RERANKER for cross-encoder model<br>• _load_reranker() function:<br>• - Loads CrossEncoder model from sentence-transformers<br>• - Caches in global variable<br>• Retriever class:<br>• - __init__: reads config flags (use_bm25, use_reranker)<br>• - Loads FAISS index, chunk metadata, BM25 corpus<br>• _load_db() method:<br>• - Reads FAISS index: faiss.read_index()<br>• - Reads chunk JSON: loads as TextChunk objects<br>• - Reads BM25 corpus: creates BM25Okapi instance<br>• _load_model() method:<br>• - Loads embedding model for query encoding<br>• _dense_search() method:<br>• - Encodes query to vector<br>• - L2 normalizes query vector<br>• - Searches FAISS: index.search()<br>• - Returns dict: {chunk_index: cosine_score}<br>• _sparse_search() method:<br>• - Tokenizes query: lowercase split<br>• - Gets BM25 scores for all documents<br>• - Sorts and returns top-N<br>• - Returns dict: {chunk_index: bm25_score}<br>• _rrf_merge() static method:<br>• - Reciprocal Rank Fusion algorithm<br>• - Converts scores to ranks (0 = best)<br>• - Formula: score = 1/(k + rank_dense) + 1/(k + rank_sparse)<br>• - k=60 is standard constant<br>• - Returns merged indices sorted by RRF score<br>• _rerank() method:<br>• - Creates (query, chunk.text) pairs<br>• - CrossEncoder predicts relevance scores<br>• - Sorts by reranker score descending<br>• search() method - full pipeline:<br>• - Step 1: Dense FAISS search (top-N candidates)<br>• - Step 2: Sparse BM25 search (top-N candidates)<br>• - Step 3: RRF merge (if BM25 enabled)<br>• - Step 4: CrossEncoder reranking (if enabled)<br>• - Step 5: Return top_k final chunks<br>• search_expanded() method:<br>• - Generates query variations with expand_query_simple()<br>• - Searches with each variation<br>• - Deduplicates results by chunk_id<br>• - Reranks merged results<br>• - Improves recall by ~30-40% |

| | |
|---|---|
| **After** | Sorted list of TextChunk objects, most relevant first. Contains text, source, page for each chunk. Passed to pedagogical_engine.py for slide generation. |

## query_expansion.py

| | |
|---|---|
| **Role** | Multi-query generation module. Creates query variations to improve retrieval coverage and recall. |
| **Before** | Single user query string (e.g., 'machine learning basics'). |
| **Inside** | • expand_query_simple() function:<br>• - Rule-based expansion (no LLM needed)<br>• - Always includes original query first<br>• - Adds 'what is {query}' if not already a definition query<br>• - Adds 'examples of {query}' if 'example' not in query<br>• - Adds '{query} concept explanation'<br>• - Returns list of 3-4 variations<br>• expand_query_llm() async function:<br>• - LLM-powered expansion for better diversity<br>• - Prompt asks for 3 alternative queries:<br>• - One more specific/technical<br>• - One broader/conceptual<br>• - Include synonyms and related terms<br>• - Expects JSON response: {queries: [...]}<br>• - Falls back to simple expansion on error<br>• - Returns max 4 queries<br>• deduplicate_results() function:<br>• - Merges results from multiple queries<br>• - Uses chunk_id or text hash for dedup<br>• - Preserves order (first occurrence wins)<br>• - Limits to top_k results |
| **After** | List of 3-4 query variations. Used by retrieval.search_expanded() to search multiple times and merge results, improving recall by 30-40%. |

## llm.py

| | |
|---|---|
| **Role** | Multi-backend LLM engine. Supports Groq API (fast cloud) and Ollama (local fallback). Includes batch generation and caching. |
| **Before** | Query string + context chunks, OR custom prompt string for direct generation. |
| **Inside** | • LLMEngine class:<br>• - __init__: reads Ollama config from config.yaml<br>• - Reads Groq config from environment variables<br>• - GROQ_API_KEY enables Groq backend<br>• - GROQ_MODEL defaults to 'llama-3.3-70b-versatile'<br>• - Creates RateLimitHandler for 429 handling<br>• _call_groq() async method:<br>• - Checks cache first (get_cached)<br>• - Builds HTTP request to Groq API<br>• - Headers: Authorization Bearer token<br>• - Payload: model, messages, temperature, max_tokens<br>• - response_format: json_object for structured output<br>• - Handles rate limiting (429):<br>• - Reads retry-after header<br>• - Waits with exponential backoff<br>• - Retries on 500, 502, 503, 504<br>• - Caches successful response<br>• _call_ollama() async method:<br>• - Checks cache first<br>• - Builds request to local Ollama API<br>• - Payload: model, prompt, stream=False<br>• - format: 'json' for JSON mode<br>• - Handles CUDA OOM: reduces context length and retries<br>• - Caches successful response<br>• generate_async() method:<br>• - Main entry point for generation<br>• - Routes to Groq or Ollama based on backend<br>• - Accepts prompt_override for custom prompts<br>• - Accepts model_override for specific models<br>• generate_all_slides_batch() method:<br>• - Generates ALL slides in ONE LLM call<br>• - Much faster than sequential (5x speedup)<br>• - Builds comprehensive prompt with:<br>• - Topic, language, num_slides<br>• - Full context from documents<br>• - Available images with descriptions<br>• - JSON schema for output<br>• - Parses JSON response<br>• - Returns list of slide dicts<br>• judge_async() method:<br>• - Optional quality scoring<br>• - Asks LLM to rate slide 1-10<br>• - Returns {score, feedback}<br>• _build_prompt() method:<br>• - Formats context chunks with bullets |

| | |
|---|---|
| | • - Adds question and instruction<br>• generate() sync method:<br>• - Wrapper for CLI usage<br>• - Runs async in event loop |
| **After** | JSON string containing slide content: title, bullets (with source_id), key_message, visual_hint, image_id, speaker_notes, quality_score. Passed to pedagogical_engine for validation. |

## llm_cache.py

| | |
|---|---|
| **Role** | Two-level caching system for LLM responses. Saves API costs and reduces latency for repeated queries. |
| **Before** | Prompt string + model identifier for cache key generation. |
| **Inside** | • Cache configuration:<br>• - MAX_MEMORY_ENTRIES: 200 (limit RAM usage)<br>• - DISK_TTL_SECONDS: 86400 (24 hours)<br>• - CACHE_DIR: data/cache/llm/<br>• _cache_key() function:<br>• - Combines model + prompt<br>• - Creates SHA-256 hash<br>• - Returns first 32 chars as key<br>• _disk_path() function:<br>• - Creates cache directory if needed<br>• - Returns path: CACHE_DIR/{key}.json<br>• get_cached() function:<br>• - L1 Check: memory cache (dict lookup)<br>• - Thread-safe with _cache_lock<br>• - Instant if hit<br>• - L2 Check: disk cache (file read)<br>• - Reads JSON file<br>• - Checks TTL (timestamp vs now)<br>• - If expired: deletes file, returns None<br>• - If valid: promotes to L1, returns response<br>• - Returns None on miss<br>• set_cached() function:<br>• - Skips empty/error responses (< 10 chars)<br>• - L1 Store: adds to memory dict<br>• - Evicts oldest if at capacity (FIFO)<br>• - L2 Store: writes JSON to disk<br>• - Includes: prompt_preview, model, response, timestamp<br>• clear_cache() function:<br>• - Clears memory dict<br>• - Deletes all .json files in cache dir<br>• - Returns count of entries removed<br>• cache_stats() function:<br>• - Returns memory_entries count<br>• - Returns disk_entries count<br>• - Returns configuration values |

| | |
|---|---|
| **After** | Cached response string on HIT (skips LLM call entirely), or None on MISS (proceeds with LLM call). Dramatically reduces costs for repeated queries. |

## retry_utils.py

| | |
|---|---|
| **Role** | Retry logic with exponential backoff. Makes API calls robust against transient failures and rate limits. |
| **Before** | Async function to execute, plus retry configuration (max_attempts, delays). |
| **Inside** | • RETRYABLE_EXCEPTIONS tuple:<br>• - ConnectionError<br>• - TimeoutError<br>• - OSError<br>• RETRYABLE_STATUS_CODES set:<br>• - 429 (Rate Limited)<br>• - 500 (Internal Server Error)<br>• - 502 (Bad Gateway)<br>• - 503 (Service Unavailable)<br>• - 504 (Gateway Timeout)<br>• RetryExhausted exception:<br>• - Raised after all attempts fail<br>• - Contains last_error and attempts count<br>• retry_async() function:<br>• - Parameters: func, max_attempts=3, base_delay=1.0, max_delay=30.0<br>• - Loop through attempts:<br>• - Try to execute function<br>• - On success: return result<br>• - On retryable exception:<br>• - Calculate delay: base * 2^attempt<br>• - Cap at max_delay<br>• - Add jitter: ±25% randomization<br>• - Log attempt number and delay<br>• - Sleep and retry<br>• - On non-retryable exception: raise immediately<br>• - After max_attempts: raise RetryExhausted<br>• with_retry() decorator:<br>• - Wraps async functions with retry logic<br>• - Usage: @with_retry(max_attempts=3)<br>• RateLimitHandler class:<br>• - Tracks consecutive rate limit hits<br>• - handle_rate_limit() method:<br>• - Increments consecutive_hits counter<br>• - Uses retry-after header if provided<br>• - Otherwise: exponential backoff based on hits<br>• - Adds jitter<br>• - Returns delay to wait<br>• - reset() method:<br>• - Called after successful request<br>• - Resets consecutive_hits to 0 |
| **After** | Successful function result after retries, or RetryExhausted exception after all attempts fail. Makes API calls resilient to network issues. |

## context_manager.py

| | |
|---|---|
| **Role** | Smart context preparation module. Formats retrieved chunks for LLM consumption with token budget optimization. |
| **Before** | List of retrieved TextChunk objects + token budget configuration. |
| **Inside** | • CHARS_PER_TOKEN constant: 4 (approximation for English)<br>• estimate_tokens() function:<br>• - Returns len(text) // 4<br>• - Quick estimate without tokenizer<br>• truncate_to_sentence() function:<br>• - If text <= max_chars: return as-is<br>• - Find last sentence boundary before limit<br>• - Looks for: '. ' or '! ' or '? ' or '.\n'<br>• - Must keep at least 50% of content<br>• - Fallback: truncate at last space + '...'<br>• format_chunk() function:<br>• - Extracts source, page, text from chunk<br>• - If include_metadata: adds [Source: X, Page Y] prefix<br>• - Returns formatted string<br>• prepare_context() function:<br>• - Parameters: chunks, max_tokens=2500, include_metadata, deduplicate<br>• - Calculates max_chars from max_tokens<br>• - Sentence-level deduplication:<br>• - Splits each chunk into sentences<br>• - Uses first 50 chars as dedup key<br>• - Skips sentences already seen<br>• - Formats chunks with metadata<br>• - Builds context respecting token budget:<br>• - If chunk fits entirely: append<br>• - If partially fits: truncate at sentence<br>• - If budget exhausted: stop<br>• - Joins with double newlines<br>• - Logs: chunks used, chars, tokens<br>• prepare_context_for_slides() function:<br>• - Scales token budget with slide count<br>• - Base: 1500 tokens<br>• - Per slide: +400 tokens<br>• - Cap at 3500 tokens<br>• - More slides = more context allowed |
| **After** | Formatted context string with source attribution, optimized for LLM token limit. Passed to pedagogical_engine for slide generation prompts. |

## pedagogical_engine.py

| | |
|---|---|
| **Role** | Core slide generation engine. Creates educational slides following pedagogical principles with quality control. |
| **Before** | Query + context chunks + num_slides + language configuration. |
| **Inside** | • SLIDE_ARC list: teaching order<br>• - intro, definition, concept, example, process<br>• - comparison, case_study, summary<br>• - Cycles for longer presentations<br>• _ONE_SLIDE_SCHEMA: JSON template shown to LLM<br>• - slide_type, title, bullets (with source_id)<br>• - key_message, visual_hint, image_id<br>• - speaker_notes, quality_score, quality_feedback<br>• _SLIDE_TYPE_SUBQUERY dict:<br>• - Maps slide types to sub-queries<br>• - 'definition' -> 'definition and meaning of {query}'<br>• - Used for targeted retrieval per slide<br>• _build_slide_prompt() function:<br>• - Builds comprehensive prompt with:<br>• - Query, context, slide_type, slide_number<br>• - Prior slide titles (to avoid repetition)<br>• - Prior visual hints (to vary diagrams)<br>• - Quality feedback from previous attempt<br>• - Available image IDs with descriptions<br>• - Rules enforced in prompt:<br>• - No content repetition<br>• - Specific facts required (not generic)<br>• - Visual hint variety<br>• _repair_json() function:<br>• - Strips markdown code blocks<br>• - Fixes Python->JSON (None->null)<br>• - Removes trailing commas<br>• - Extracts JSON from surrounding text<br>• _extract_slide_json() function:<br>• - Tries json.loads() first<br>• - Falls back to _repair_json()<br>• - Returns parsed dict or None<br>• _slide_fingerprint() function:<br>• - Creates MD5 hash of bullet texts<br>• - Used for deduplication<br>• PedagogicalEngine class:<br>• - __init__: reads config, creates LLMEngine<br>• _generate_one_slide() async method:<br>• - Up to 3 attempts per slide<br>• - Builds prompt with quality feedback<br>• - Calls LLM generate_async()<br>• - Extracts and validates JSON<br>• - Quality gate: checks quality_score<br>• - If < threshold: retry with feedback<br>• - Fingerprint dedup: checks against prior_fps |

| | |
|---|---|
| | • - If duplicate: retry<br>• - LLM-as-judge (if enabled):<br>• - Second LLM scores the slide<br>• - If judge score < threshold: retry<br>• - Returns accepted slide dict<br>• generate_lesson_async() method:<br>• - Determines language label<br>• - Routes to parallel or sequential mode<br>• _generate_parallel() method:<br>• - Creates coroutine for each slide<br>• - Runs all with asyncio.gather()<br>• - Post-dedup pass removes duplicates<br>• - Total time: ~1x LLM latency<br>• _generate_sequential() method:<br>• - Generates slides one at a time<br>• - Passes prior_titles/hints to each call<br>• - Better quality control, slower |
| **After** | Dict containing: topic (string), slides (list of validated slide dicts). Each slide has title, bullets, key_message, visual_hint, etc. Passed to slide_generator.py. |

## schemas.py

| | |
|---|---|
| **Role** | Pydantic models for structured validation. Ensures LLM output conforms to expected schema. |
| **Before** | Raw dict from LLM JSON output (potentially malformed or incomplete). |
| **Inside** | • SlideType literal:<br>• - Valid values: title, intro, definition, concept<br>• - example, comparison, process, case_study, summary<br>• VisualHint literal:<br>• - Valid values: flowchart, mindmap, timeline<br>• - comparison, process, hierarchy, none<br>• BulletPoint model:<br>• - text: str (10-500 chars)<br>• - source_id: Optional[str] (max 100 chars)<br>• - @validator: strips whitespace<br>• SlideSchema model:<br>• - slide_type: defaults to 'concept'<br>• - title: str (3-100 chars)<br>• - bullets: list[BulletPoint] (1-6 items)<br>• - key_message: Optional[str] (max 200)<br>• - visual_hint: defaults to 'none'<br>• - image_id: Optional[str]<br>• - speaker_notes: Optional[str] (max 500)<br>• - quality_score: int (1-10)<br>• Field validators:<br>• - clean_title: strips whitespace<br>• - normalize_slide_type: lowercases, validates<br>• - normalize_visual_hint: lowercases, validates<br>• - parse_quality_score: converts to int, clamps 1-10<br>• - normalize_bullets: handles various formats<br>• - Dict with 'text' key: use as-is<br>• - Dict with 'content' or 'bullet': convert<br>• - Plain string: wrap in dict<br>• to_dict() method:<br>• - Converts Pydantic model to plain dict<br>• LessonSchema model:<br>• - topic: str (3-200 chars)<br>• - slides: list[SlideSchema]<br>• validate_slide() function:<br>• - Strict validation<br>• - Returns (SlideSchema, None) on success<br>• - Returns (None, error_message) on failure<br>• validate_and_fix_slide() function:<br>• - Lenient validation with auto-fixing<br>• - Ensures title exists (default: 'Untitled')<br>• - Converts string bullets to list<br>• - Filters short bullets (< 10 chars)<br>• - Adds default bullet if empty<br>• - Returns valid dict even from bad input |
| **After** | Validated SlideSchema objects or fixed dicts with guaranteed structure. Ensures downstream code receives consistent data. |

## slide_generator.py

| | |
|---|---|
| **Role** | Converts lesson plan dicts to SlideData dataclass objects for rendering. |
| **Before** | Lesson dict from pedagogical_engine containing topic and slides array. |
| **Inside** | • VALID_SLIDE_TYPES set:<br>• - title, definition, concept, example<br>• - comparison, summary, content, section<br>• SlideData dataclass:<br>• - slide_type: str<br>• - title: str<br>• - bullets: list (default empty)<br>• - speaker_notes: str (default empty)<br>• - key_message: str (default empty)<br>• - visual_hint: str (default 'none')<br>• - image_id: str or None<br>• - quality_score: int (default 8)<br>• - quality_feedback: str (default empty)<br>• build_slides() function:<br>• - Extracts topic from lesson dict<br>• - Gets raw_slides list<br>• - Creates title slide first:<br>• - slide_type = 'title'<br>• - title = topic<br>• - speaker_notes = 'Introduction to: {topic}'<br>• - For each raw slide:<br>• - Extracts and normalizes bullets<br>• - Normalizes slide_type (lowercase, validate)<br>• - Falls back to 'concept' if invalid<br>• - Creates SlideData with all fields<br>• - Logs: 'Built N slides (1 title + N-1 content)' |
| **After** | List of SlideData dataclass objects. Passed to diagram_generator.py for diagram creation and html_renderer.py for final output. |

## diagram_generator.py

| | |
|---|---|
| **Role** | Generates Mermaid diagrams based on slide visual_hint. Creates flowcharts, mindmaps, timelines, etc. |
| **Before** | List of SlideData objects with visual_hint and bullets fields. |
| **Inside** | • _safe() function:<br>• - Escapes special characters for Mermaid<br>• - Replaces: quotes, newlines, brackets, <>, \|, #, &<br>• - Truncates to max length (default 120)<br>• _to_diagram_labels() function:<br>• - Converts bullet text to SHORT labels<br>• - Removes markdown bold/italic<br>• - Extracts keyword before colon or hyphen<br>• - Falls back to first 4 words<br>• - Deduplicates labels<br>• - Returns list of clean keywords<br>• _mermaid_flowchart() function:<br>• - Creates: flowchart TD<br>• - Assigns node IDs: A, B, C...<br>• - Adds arrows: A --> B --> C<br>• _mermaid_mindmap() function:<br>• - Creates: mindmap<br>• - Root node from key_message or title<br>• - Child nodes from bullet labels<br>• _mermaid_timeline() function:<br>• - Creates: timeline<br>• - Title from slide title<br>• - Phases from bullet labels<br>• _mermaid_comparison() function:<br>• - Creates two-branch flowchart<br>• - Splits bullets into left/right<br>• - ROOT --> L, ROOT --> R<br>• _mermaid_process() function:<br>• - Flowchart with cycle arrow<br>• - Last node links back to first<br>• - Shows iterative process<br>• _mermaid_hierarchy() function:<br>• - Tree structure<br>• - ROOT fans out to all children<br>• _flow_png() function:<br>• - Matplotlib fallback for PNG<br>• - Creates boxes with arrows<br>• - Dark theme styling<br>• generate_all_diagrams() function:<br>• - Iterates through slides<br>• - Skips title slides<br>• - Skips slides with image_id (already has visual)<br>• - Skips slides with hint='none'<br>• - Avoids repeating same diagram type consecutively<br>• - Routes to appropriate Mermaid function |

| | |
|---|---|
| | • - Returns dict: {slide_index: {mermaid, png}} |
| **After** | Dict mapping slide indices to diagram data: {mermaid: 'flowchart TD...', png: '/path/to/fallback.png'}. Embedded in HTML output by html_renderer.py. |

## visual_strategy.py

| | |
|---|---|
| **Role** | Intelligent visual assignment module. Decides WHEN to include diagrams or images based on content analysis. |
| **Before** | Slide dict with title, bullets, slide_type, and visual_hint fields. |
| **Inside** | • NO_VISUAL_TYPES set: {'title'}<br>• - Title slides never get visuals<br>• MINIMAL_VISUAL_TYPES set: {'intro', 'summary', 'definition'}<br>• - These rarely need visuals<br>• DIAGRAM_KEYWORDS set:<br>• - process, workflow, steps, flow, pipeline<br>• - architecture, structure, system<br>• - hierarchy, tree, organization<br>• - comparison, vs, versus, compare<br>• - timeline, evolution, history, phases<br>• - cycle, loop, iterative<br>• - relationship, connection, interaction<br>• - layer, stack, component<br>• IMAGE_KEYWORDS set:<br>• - example, case study, illustration<br>• - screenshot, photo, picture, visual<br>• - interface, ui, dashboard, screen<br>• NO_VISUAL_KEYWORDS set:<br>• - definition, meaning, what is, overview<br>• - introduction, summary, conclusion<br>• - key points, takeaways, recap<br>• decide_visual() function:<br>• - Rule 1: title slides -> 'none'<br>• - Rule 2: check NO_VISUAL_KEYWORDS<br>• - Rule 3: intro/summary need strong signal<br>• - Count diagram keywords in content<br>• - Need >= 3 matches for diagram<br>• - Rule 4: respect visual_hint from LLM<br>• - Validate against diagram keywords<br>• - Reject if no keyword support<br>• - Rule 5: check image_id presence<br>• - Rule 6: auto-detect based on keywords<br>• - Rule 7: default to 'none' for facts<br>• validate_image_relevance() function:<br>• - Compares slide content to image context<br>• - Removes stop words<br>• - Calculates Jaccard similarity<br>• - Boosts score if title words overlap<br>• - Returns True if >= threshold |

| | |
|---|---|
| | • filter_slides_for_diagrams() function:<br>• - Scores all slides for diagram potential<br>• - Sorts by score descending<br>• - Limits to max 3 diagrams (or 40% of slides)<br>• - Prevents visual overload |
| **After** | VisualDecision string: 'diagram', 'image', 'both', or 'none'. Used to control diagram generation and image assignment in the pipeline. |

## html_renderer.py

| | |
|---|---|
| **Role** | Professional HTML presentation renderer. Creates beautiful, animated presentations with multiple themes. |
| **Before** | SlideData objects + diagram map + image map + theme selection. |
| **Inside** | • THEMES dict - 8 color schemes:<br>• - obsidian: Dark purple/blue gradient<br>• - aurora: Dark green/cyan<br>• - inferno: Dark orange/red<br>• - ocean: Dark blue<br>• - forest: Dark green<br>• - neon: Dark pink/cyan<br>• - galaxy: Dark purple/pink<br>• - cream: Light beige (only light theme)<br>• Each theme defines:<br>• - bg, bg2: background colors<br>• - surf, brd: surface and border colors<br>• - ink, muted: text colors<br>• - a, a2, a3: accent colors<br>• - orb1, orb2, orb3: particle colors<br>• - dark: boolean flag<br>• _PPTX_TO_HTML_THEME: maps friendly names<br>• _auto_theme() function:<br>• - Hashes presentation ID<br>• - Selects consistent theme<br>• _esc() function:<br>• - HTML entity escaping<br>• - Handles: &, <, >, quotes, newlines<br>• _esc_mermaid() function:<br>• - Minimal escaping for Mermaid<br>• - Preserves --> arrows<br>• _rgb() function: hex to RGB tuple<br>• _img_style() function:<br>• - Creates background-image CSS<br>• - Adds overlay div for readability<br>• _chart_js() function:<br>• - Generates Chart.js code<br>• - Auto-detects chart type:<br>• - 'trend', 'growth' -> line chart<br>• - <= 4 items or 'share' -> doughnut<br>• - default -> bar chart<br>• - Gradient fills matching theme<br>• - Animated reveal<br>• render() function:<br>• - Main entry point<br>• - Creates output directory<br>• - Generates unique filename<br>• - Builds complete HTML document:<br>• - DOCTYPE and head with fonts<br>• - CSS variables from theme |

| | |
|---|---|
| | • - Particle.js background<br>• - Slide container with navigation<br>• - Each slide with content, diagrams, images<br>• - Mermaid.js integration<br>• - Keyboard navigation (arrows)<br>• - Speaker notes panel<br>• - Writes HTML file<br>• - Returns file path |
| **After** | Complete HTML file saved to outputs/ directory. Contains all CSS, JS, and content inline. Ready to view in browser or serve via /view endpoint. |

## evaluation.py

| | |
|---|---|
| **Role** | RAG pipeline quality metrics. Measures retrieval relevance, diversity, and generation groundedness. |
| **Before** | Query string + retrieved chunks + generated text for evaluation. |
| **Inside** | • Global: _EVAL_MODEL for embedding model cache<br>• RetrievalMetrics dataclass:<br>• - query: string<br>• - num_chunks: int<br>• - avg_relevance: float (0-1)<br>• - max_relevance: float<br>• - min_relevance: float<br>• - diversity_score: float (0-1)<br>• - coverage_estimate: float (0-1)<br>• - summary() method: emoji + key stats<br>• GenerationMetrics dataclass:<br>• - groundedness: float (answer based on context?)<br>• - relevance: float (answer matches query?)<br>• - completeness: float (answer is thorough?)<br>• RAGEvaluator class:<br>• _cosine_similarity() method:<br>• - Computes cosine between two vectors<br>• - Handles zero vectors<br>• _pairwise_diversity() method:<br>• - Computes all-pairs similarity<br>• - Returns 1 - average similarity<br>• - High = diverse chunks<br>• evaluate_retrieval() method:<br>• - Embeds query and all chunks<br>• - Computes query-chunk similarities<br>• - Calculates avg/max/min relevance<br>• - Computes diversity score<br>• - Estimates coverage from unique words<br>• evaluate_generation() method:<br>• - Embeds query, context, generated text<br>• - Groundedness: generated vs context similarity<br>• - Relevance: generated vs query similarity |

| | |
|---|---|
| | • - Completeness: word coverage + length score<br>• evaluate_full_pipeline() method:<br>• - Combines retrieval + generation metrics<br>• - Weighted overall score:<br>• - 30% retrieval relevance<br>• - 10% diversity<br>• - 30% groundedness<br>• - 20% relevance<br>• - 10% completeness<br>• - Letter grade: A (>0.6), B, C, D |
| **After** | Metrics dicts with scores and summaries. Exposed via POST /evaluate endpoint for debugging and optimization. |

## health.py

| | |
|---|---|
| **Role** | System health monitoring module. Checks status of all components for debugging and monitoring. |
| **Before** | None (reads system state from various modules). |
| **Inside** | • check_faiss_index() function:<br>• - Reads index path from config<br>• - Checks if .index and .json files exist<br>• - Loads FAISS index, counts vectors<br>• - Loads JSON, counts chunks<br>• - Checks if .bm25.json exists<br>• - Returns: status, vectors, chunks, bm25_ready<br>• check_embedding_model() function:<br>• - Gets model name and device from config<br>• - Checks if global model is loaded<br>• - Returns: model, device, status, dimensions<br>• check_llm_backend() function:<br>• - Checks GROQ_API_KEY environment variable<br>• - If set: backend='groq', status='configured'<br>• - Else: backend='ollama'<br>• - Pings Ollama /api/tags endpoint<br>• - status='connected' or 'unreachable'<br>• check_reranker() function:<br>• - Checks config use_reranker flag<br>• - Checks if global reranker model loaded<br>• - Returns: enabled, model, status<br>• check_cache() function:<br>• - Calls cache_stats() from llm_cache<br>• - Returns memory/disk entry counts<br>• check_history() function:<br>• - Calls load_history()<br>• - Counts presentations<br>• - Returns: total_presentations, file_exists<br>• check_ocr() function:<br>• - Gets engine type from config<br>• - Checks if EasyOCR is pre-loaded<br>• - Returns: engine, languages, status<br>• get_system_info() function:<br>• - Python version<br>• - Platform (Windows/Linux/Mac)<br>• - Architecture<br>• - CUDA availability and device name<br>• - GPU memory<br>• get_config_summary() function:<br>• - Key config values for display<br>• full_health_check() function:<br>• - Combines all checks<br>• - Returns timestamp and version<br>• - system, config, components sections<br>• quick_status() function: |

| | |
|---|---|
| | • - Lightweight check<br>• - Only FAISS and LLM backend<br>• - For frequent polling |
| **After** | JSON dict with comprehensive system status. Exposed via GET /health and GET /health/quick endpoints. |

## history_store.py

| | |
|---|---|
| **Role** | Persistent presentation history. Saves metadata for all generated presentations. |
| **Before** | Presentation metadata: path, prompt, topic, slides, theme, model. |
| **Inside** | • HISTORY_FILE: outputs/presentation_history.json<br>• _history_lock: threading.Lock for thread safety<br>• _load() function:<br>• - Checks if HISTORY_FILE exists<br>• - Reads and parses JSON<br>• - Returns list of records or empty list<br>• _save() function:<br>• - Creates parent directory if needed<br>• - Writes JSON with indent=2<br>• - UTF-8 encoding<br>• record_presentation() function:<br>• - Creates entry dict:<br>• - id: session_id or auto-generated<br>• - created_at: ISO timestamp<br>• - prompt: user query<br>• - topic: extracted topic<br>• - num_slides: count<br>• - theme: theme name<br>• - model: LLM backend used<br>• - html_path: output file path<br>• - slides: slide content (optional)<br>• - Acquires lock<br>• - Loads existing records<br>• - Inserts new entry at position 0 (newest first)<br>• - Saves updated list<br>• - Logs update<br>• load_history() function:<br>• - Acquires lock<br>• - Returns all records<br>• clear_history() function:<br>• - Acquires lock<br>• - Deletes HISTORY_FILE if exists<br>• - Logs clear action |
| **After** | JSON array in outputs/presentation_history.json. Enables history browsing in frontend via GET /history endpoint. |

# 5. Complete End-to-End Flow

Here is the complete data flow from PDF upload to HTML presentation:

## Step 1: Document Upload

User uploads PDF via React frontend. File sent to POST /generate-stream endpoint.

## Step 2: File Processing (api.py)

API saves file to temp directory, creates session ID, starts SSE stream.

## Step 3: Ingestion (ingestion.py)

PyMuPDF extracts text and images from each page. Creates DocumentPage objects.

## Step 4: OCR if Needed (ocr.py)

Scanned pages with no text go through EasyOCR. Neural network extracts text from images.

## Step 5: Chunking (text_processing.py)

Text is split into ~512 character chunks. Respects paragraph and sentence boundaries. Adds overlap for context.

## Step 6: Embedding (embeddings.py)

SentenceTransformer converts each chunk to a 768-dimensional vector. Semantic deduplication removes near-duplicates.

## Step 7: Indexing

Vectors stored in FAISS IndexFlatIP. Tokenized corpus saved for BM25. Files: .index, .json, .bm25.json

## Step 8: User Query

User types topic (e.g., 'Explain neural networks'). Optional: language, num_slides, theme.

## Step 9: Retrieval (retrieval.py)

Hybrid search: FAISS semantic + BM25 keyword. RRF fusion merges results. CrossEncoder reranks top-N. Returns most relevant chunks.

## Step 10: Context Preparation (context_manager.py)

Chunks formatted with source metadata. Token budget enforced. Sentence-level deduplication.

## Step 11: Slide Generation (pedagogical_engine.py + llm.py)

LLM generates slides following SLIDE_ARC order. Quality gate rejects low-quality slides. Fingerprint dedup prevents repetition. Batch mode: ALL slides in ONE call (5x faster).

## Step 12: Validation (schemas.py)

Pydantic validates each slide. Fixes common issues. Ensures consistent structure.

## Step 13: Diagram Generation (diagram_generator.py)

Based on visual_hint, creates Mermaid diagrams. Flowcharts, mindmaps, timelines, etc.

## Step 14: Image Assignment (api.py)

_assign_fallback_images() matches PDF images to slides based on content similarity.

## Step 15: HTML Rendering (html_renderer.py)

Combines slides, diagrams, images, theme. Outputs professional HTML with animations and speaker notes.

## Step 16: History Recording (history_store.py)

Presentation metadata saved to JSON. Enables browsing past presentations.

## Step 17: Response

SSE stream sends 'done' event with session_id. Frontend calls /view/{session_id} to display.

# 6. Key Technical Concepts

## 6.1 RAG (Retrieval-Augmented Generation)

RAG combines retrieval (finding relevant documents) with generation (LLM creating content). Instead of relying on LLM memory alone, we inject relevant context from your documents. This prevents hallucination and enables domain-specific knowledge.

## 6.2 Embeddings

Text embeddings convert words/sentences into numerical vectors (768 dimensions). Similar meanings = close vectors in space. Enables semantic search beyond keyword matching. Example: 'car' and 'automobile' have similar vectors.

## 6.3 FAISS

Facebook AI Similarity Search. Efficient nearest-neighbor search for vectors. IndexFlatIP uses inner product (equivalent to cosine after L2 normalization). In-memory, fast for thousands of vectors.

## 6.4 BM25

Best Match 25. Keyword-based ranking algorithm (TF-IDF variant). Scores documents by term frequency weighted by rarity. Catches exact matches that embeddings miss (acronyms, proper nouns, technical terms).

## 6.5 Reciprocal Rank Fusion (RRF)

Merges two ranked lists. Formula: score = 1/(k + rank_dense) + 1/(k + rank_sparse). k=60 is standard constant. Combines semantic and keyword results without score normalization issues.

## 6.6 Cross-Encoder Reranking

More accurate than cosine similarity. Reads (query, passage) together as input. Outputs relevance score. Slower, so only runs on top-N candidates (not entire index).

## 6.7 Batch Generation

Instead of N separate LLM calls (one per slide), generate ALL slides in ONE call. Reduces total time from N x latency to ~1 x latency. 5x speedup for typical presentations.

## 6.8 SSE (Server-Sent Events)

One-way streaming from server to client. Used for real-time progress updates during generation. Events: status (step updates), slide (each slide as generated), done (completion), error.

# 7. API Endpoints Reference

| Method | Endpoint | Description |
|---|---|---|
| GET | / | Health check, returns version and LLM backend |
| GET | /themes | List available presentation themes |
| **POST** | **/generate-stream** | **MAIN - Generate presentation with SSE streaming** |
| GET | /view/{session_id} | View/download generated HTML presentation |
| GET | /history | Get all past presentations |
| DELETE | /history | Clear presentation history |
| GET | /health | Full system health check (all components) |
| GET | /health/quick | Lightweight status for frequent polling |
| GET | /cache/stats | LLM cache statistics |
| DELETE | /cache | Clear LLM response cache |
| POST | /evaluate | Evaluate RAG quality for a query |

*--- End of Documentation ---*